



哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

第九届全国大学生计算机系统能力培养大赛

“龙芯杯”竞赛总结报告（团队赛）

Cache-HIT-100

指令集: LoongArch32R

队 长: 朱炫翰（前端）

队 员: 蔡源航（后端）

房效民（Cache、MMU）

林亮宇（外设、系统展示）

哈尔滨工业大学（深圳）

2025 年 8 月

目 录

目 录	I
第 1 章 概 述	4
1.1 初赛/决赛指标	4
1.1.1 功能	4
1.1.2 性能测试	4
1.1.3 系统功能及外设	4
1.2 设计概述	4
1.2.1 设计简介	4
1.2.2 设计架构图	4
第 2 章 系统详细设计	5
2.1 系统整体架构设计	5
2.2 前端设计	5
2.2.1 分支预测器	5
2.2.2 取指目标队列 FTQ.....	8
2.2.3 取指令单元与取指流水	8
2.3 指令缓冲（IB）模块设计	9
2.4 后端设计	9
2.4.1 译码模块设计（ID）	9
2.4.2 重命名映射表模块设计（RAT）	9
2.4.3 重命名模块设计（RA）	10
2.4.4 发射模块设计（ISS）	10
2.4.5 执行模块设计（EXE）	11

2.4.6 重排序缓存模块设计 (ROB)	11
2.4.7 提交模块设计 (COMMIT)	11
2.5 转接桥模块 (AXI_bridge)	12
2.6 cache 设计	13
2.6.1 icache 设计	13
2.6.2 dcache 设计	14
2.7 地址翻译设计	15
2.7.1 Addr_trans 模块:	15
2.7.2 Tlb 模块:	15
2.8 SoC 设计	15
2.8.1 地址分配	16
2.8.2 外设控制器	17
2.9 VGA 控制器.....	18
2.9.1 VGA 控制器设计.....	18
2.9.2 VGA 控制器展示.....	19
2.10 LCD 控制器	20
2.10.1 LCD 控制器设计	20
2.10.1 LCD 控制器系统展示	22
2.11 micropython 移植.....	25
第 3 章 关键技术及特色/亮点	26
第 4 章 调试及优化	27
4.1 测试方法	27
4.1.1 备赛初期	27
4.1.2 备赛中后期	27
4.1.3 在线评测	28
4.2 调试和优化过程	28
4.3 系统软件调试	29

4.3.1 调试中遇到的问题和解决办法	29
4.3.2 调试工具	34
第 5 章 关键问题及解决方案	37
第 6 章 总结与展望	40
6.1 竞赛总结	40
6.2 未来展望	43
6.3 备赛经验及建议	43

第 1 章 概 述

1.1 初赛/决赛指标

1.1.1 功能

能通过 chiplab 的 fun19 全部 81 个功能测试点

1.1.2 性能测试

初赛发布包得分 2.264 提交频率 75M

复赛发布包得分 0.936 提交频率 75M

1.1.3 系统功能及外设

能成功启动 linux 系统

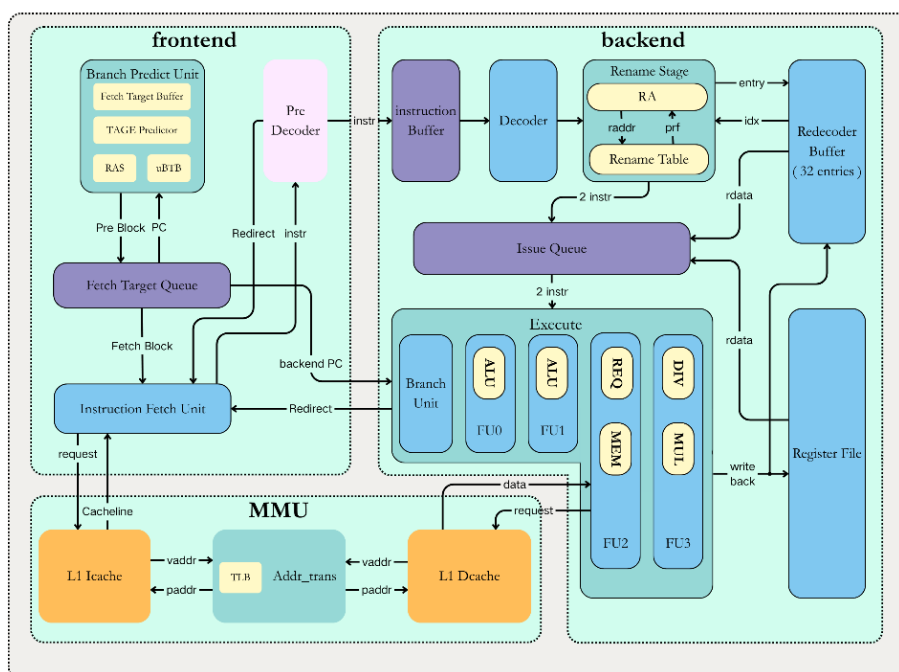
可以驱动 PS/2 键盘、VGA 显示器、驱动 LCD 屏幕进行显示和触控

1.2 设计概述

1.2.1 设计简介

本项目使用 system verilog 实现了基于 LoongArch32R 指令集的超标量动态双发射处理器。实现了除 ibar, dbar, preld 以外的所有 LA32R 指令。采用解构前端设计，拥有二级分支预测单元，具备 16KB 大小一级指令缓存和 16KB 大小一级数据缓存，能通过 AXI 接口与各个外设进行通信。

1.2.2 设计架构图



第2章 系统详细设计

2.1 系统整体架构设计

前端可大致划分为分支预测模块和取指模块，分支预测模块包含 FTB、TAGE、RAS、预译码等模块，取指模块取指宽度为 4，每周期最多向指令缓冲填入 4 条指令。由于采用了精度较高的 TAGE 分支预测器，前端的预测准确率能达到 95% 左右（单次 bitcount test 测试结果）前端各个组件的作用和交互时序将在下面介绍。

后端从流水线阶段上看大致分为译码阶段、重命名阶段、发射阶段、执行阶段和提交阶段，从功能上看包含译码器（decoder）、指令缓存队列（IB）、重命名映射表（RAT）、指令发射队列（IQ）、功能执行单元（FU）、分支跳转单元（BU）、乘法器、除法器 and 重排序缓存（ROB）。指令执行过程大致如下：1.译码阶段，从 IB 中取出两条指令进行初步译码；2.重命名阶段，指令查询 RAT，记录源寄存器对应的 ROB 索引，同时将目的寄存器对应的 ROB 索引写入 RAT。重命名结束后将指令填入 IQ 和 ROB；3.发射阶段，从 IQ 中顺序取出两条指令，根据指令记录的重命名关系，查询 ROB 确认数据是否准备完毕，当数据准备完毕且可以正常流入下一个流水级时，将指令发射出去；4.执行阶段，指令在对应的 FU 中进行相应的算术、访存操作。指令执行完毕后，将数据存入 ROB 中等待退休；5.提交阶段，检查 ROB 队列顶部的两条指令是否满足退休条件，满足则让指令进行退休，进行相应的写寄存器堆、写 csr、异常处理等操作。

2.2 前端设计

处理器前端采用取基本块结构，主要包括分支预测器（BPU），取指目标队列（FTQ）和取指令单元（IFU），预译码模块（Predecoder）。

2.2.1 分支预测器

分支预测器每周期最多对一个地址进行预测，生成的预测块最多包含 4 条指令，出于取指单元设计复杂度考虑，预测块会在跨行处截断。

分支预测器核心组件包括 uBTB、FTB、TAGE，实现了条件分支指令、函数调

用指令、无条件分支指令的取指跳转地址的预测（通过 FTB 存储的跳转目标实现）；实现取指宽度的计算（只取到第一条预测发生跳转的指令，通过 FTB 实现）；实现对条件跳转指令跳转方向的预测（通过 TAGE 实现）。

2.2.1.1 BPU 流水

L0:

使用当前 PC 查询 uBTB、FTB 和 TAGE 预测器。

uBTB 当拍返回，生成发送给 FTQ 的基础预测块。

L1:

L0 生成的预测块发送至 FTQ。

FTB 和 TAGE 查询返回，如果命中且与 uBTB 产生的预测块有差异，生成主预测器重定向信号和新的预测块。

2.2.1.2 取指目标缓冲 FTB:

FTB 功能:

用于存储含有分支的预测块信息，存储的信息较多，使用 BRAM 实现以优化板上资源。规格为 4 路组相连，每组 1K 项，使用 pc 的第 3 至 12 位进行索引，pc 的 13 至 32 位作为 tag 精确匹配。通过预测块的开始地址来查询 FTB 的表项。

FTB 条目内容:

tag（由预测块开始地址决定）

fall-through address（分支不跳转情况下的顺序执行地址）

jump-target address（分支跳转目标地址）

branch-type（分支指令类型）

FTB 更新的时机:

因为 FTB 中的信息是整个前端推测执行的基础，必须保证其正确性，所以只有当分支指令所在的基本块在后端进行提交时才将分支信息推入 FTB 进行更新。

FTB 更新的条件:

当基本块中出现，非条件分支、条件分支第一次跳转、分支指令跳转信息错误这三种情况中的一种时，将基本块作为 FTB 的一个条目写入 FTB 或者对 FTB 条

目的信息进行更新。

2.2.1.3 简单预测器 uBTB:

当拍返回的简易预测器, 提供基础无空泡预测块(默认取指宽度 4, 跨行截断)

由于 FTB 只有在基本块提交时才进行信息的更新, 当分支指令在后端向前端发出重定向后, 还需要至少 3 个周期才能通过后端提交更新到前端 FTB 中。对于来回跳转的小循环, 由于 BPU 的预测流水线速度很快, 容易出现 FTB 信息还未更新, BPU 就已经对发出重定向的分支指令进行预测的情况, 也就是主预测器会在循环中丢失一次预测。

针对这个现象, uBTB 被设计为一个只有 16 个条目, 采用 pc 第 3 至 6 位作为索引, 高 26 位作为 tag 精确匹配的缓冲。当后端向前端发出分支重定向时, uBTB 立刻根据分支跳转的目标 pc 和分支指令 pc 判断, 如果是来回跳转的分支, 就向 uBTB 中填入一个有效条目。

当 uBTB 条目命中时, 会根据当拍返回的跳转目标产生一个跳转的预测。如果 FTB 中存有同一条跳转指令的 pc, 那么下周期 FTB 命中, 将根据主预测器给出的预测信息覆盖上一周期 uBTB 产生的基础预测。

2.2.1.4 返回地址栈 RAS:

返回地址栈分为两个栈, 32 个条目, 分别由前端和后端各自维护。

当 FTB 命中了一条调用指令时, 主预测器将调用入口地址作为预测目标, 同时将调用指令的返回地址压入前端维护的 RAS。当 FTB 命中了一条返回指令时, 从 RAS 中取出返回地址。

后端如果执行了调用指令或者返回指令, 会自动维护 RAS, 保证后端维护的栈一定是正确的。如果后端向前端发出重定向, 那么用后端维护的 RAS 去恢复前端的 RAS。当同一个函数在不同地方进行调用时, 前端总是能对函数调用的返回做出正确的预测。

2.2.1.5 主预测器 TAGE

TAGE 预测器是专门用于预测条件分支指令跳转方向的二级预测器表, 其预测结果需要在下一拍才能返回。该预测器的规格如下: 基础预测器包含 8K 项, 每

个项配备 2 比特计数器；标记预测器则为 1K 项，每个项包含 12 比特 tag、3 比特计数器和 3 比特可用位。基础预测器通过 PC 索引不同项的饱和计数器来实现分支方向预测，而 4 个标记预测器各自采用了不同的全局分支历史长度，它们通过 PC 全局历史的摘要进行异或运算来索引 3 比特饱和计数器，同时借助 tag 比对机制减少哈希冲突，提升预测精度。

2.2.2 取指目标队列 FTQ

FTQ 模块的主要功能是作为分支预测模块与取指模块之间的缓冲区，负责存储预测信息，并且接收来自后端提交的训练数据，送回用于分支预测的训练。

当 BPU 模块接收到来自 PC 的查询请求时，在默认预测不发生跳转的情况下，会在当拍将预测取指块发送至 FTQ 中，确保取指流程的顺畅进行。如果下个周期主预测器给出有效预测，会将上个周期发送至 FTQ 的取指块修改。在我们的 CPU 中，FTQ 队列的大小是 8，考虑到每一个块最多有 4 条指令，所以前端最多等待后端提交 32 条指令，这与后端 ROB 条目的数量是相同的。

2.2.3 取指令单元与取指流水

IFU 每周期从 FTQ 中读取预测块，进行地址翻译，如果没有出现相关异常，那么向 Icache 发出取指令请求。

取指流水：

L0 读预测块：

从 FTQ 中读出预测块，将预测块起始地址送入地址翻译单元进行翻译，当拍接受地址翻译的返回结果并存入寄存器中。

L1 发取指请求：

对 L0 地址翻译的结果进行处理，使用物理地址向 Icache 发送取指请求，如果出现异常则不进行取指。

L2 等待指令返回：

等待 Icache 将指令返回。由于 Icache 可能在不定期返回指令，因此前端维护了一个和 Icache 返回指令有效同拍的信号表明读事务有没有完成。当后端发出

冲刷信号时，L2 根据是否有读事务正在进行，决定 Icache 接下来第一个返回的数据是否接收，同时前端将所有寄存器和输出置空，直至有效的指令被取回。

L3 分割指令，预译码重定向：

对返回的 cacheline 进行切割，将分割完成后的指令送入预译码器，对不依赖寄存器值的强制分支指令进行译码，如果前端发现没有对这些指令进行预测，将发出预译码重定向信号冲刷前端取指和预测流水。

2.3 指令缓冲（IB）模块设计

使用指令缓冲来实现前后端的解耦，从而使前端取指和后端执行互不干扰。在模块中实例化一个 FIFO 作为指令缓冲队列，从前端取回 1-4 条指令存入队列中，执行时取出两条指令进行译码。FIFO 中的一条信息包含了：指令有效位、指令的 PC 值、指令的二进制码、分支预测的信息以及例外信息。队列的判满逻辑采用灵活判满，即根据前端取回的指令数量判断队列是否为满。当队列满时，会暂停前端取指，当队列空时，向译码阶段发送的指令有效位为 0，即相当于发射空指令，以此来保证前后端工作的独立性。

2.4 后端设计

2.4.1 译码模块设计（ID）

译码模块每周期从指令缓冲中取出两条指令进行译码，实例化两个译码器，分别对两条指令译码产生执行过程所需的控制信号、立即数。在该模块检测指令不存在例外，并将此例外信息附在指令上，和指令 pc 值、分支预测信息、控制信号、立即数等作为译码结果传递至下个流水级。

2.4.2 重命名映射表模块设计（RAT）

该模块拥有 4 个读端口、2 个写端口和 2 个无效化端口，内部存储分为两个部分：1.32 位的寄存器 valid，每一位对应一个逻辑寄存器的映射关系是否有效； $2.32 \times \log_2(\text{ROB_DEPTH})$ 的寄存器数组 prf，每一项记录着一个逻辑寄存器映射到的 ROB 索引。

在重命名阶段，若指令需要写寄存器堆，则以其目的寄存器为 RAT 索引，将对应位的 valid 置 1，将对应位的 prf 记为该指令所在的 ROB 索引；若指令需要使用源寄存器的值，则以其源寄存器为 RAT 索引，读取对应的 valid 和 prf 值，供发射时使用。需要注意的是当两条指令之间存在写后读冒险时，后一条指令应直接将前一条指令的 ROB 索引作为源寄存器的 prf 值，且 valid 为 1。

在提交阶段，当指令退休时，需要以其目的寄存器为 RAT 索引，如果对应位的 valid 值为 1 且 prf 值等于该指令的 ROB 索引，意味着这一映射关系一直存在，没有被后面指令的重命名所修改，然后就将对应位的 valid 置 0 即可。

2.4.3 重命名模块设计（RA）

指令进入重命名模块后，需要查询重命名映射表和重排序缓存完成重命名。指令先查询重排序缓存，获得指令将要被放入的 ROB 索引。接着如上一部分所述查询和写入重命名映射表，获得指令源寄存器的映射关系。最后准备好放入 IQ 和 ROB 的数据，在 IQ 和 ROB 都未滿时将数据送入 IQ 和 ROB。

2.4.4 发射模块设计（ISS）

发射模块每周期从 IQ 中顺序取出两条指令，先根据指令类型判断能否进行双发射：1.两条指令发生写后读冒险；2.同时有两条访存指令 3.同时有两条乘法指令；4.前一条指令为分支跳转指令；5.两条指令中存在特权指令；6.前一条指令发生了例外；7.前一条指令被打上重取标记时，进行单发射，其他情况在 FU 使用不冲突的情况下进行双发射。

接着判断一条指令能否发射，主要是看其源操作数是否已经准备好。指令根据其源寄存器的映射关系查找 ROB，ROB 返回 complete 信号（表示数据是否准备好）以及 32 位数据。数据准备好后根据指令类型和 FU 的使用情况将指令发射到相应的 FU 中执行。

在部分情况下需要阻塞指令的发射：1.前面指令的执行过程中发现了例外和重取信号；2.BU 中的分支跳转指令发现需要进行重定向；3.已经发射且未提交的指令中存在写 csr 的指令。

2.4.5 执行模块设计（EXE）

执行模块内部实例化了 4 个不同的 FU 和 1 个 BU 作为指令执行单元。FU0 和 FU1 作为普通指令的执行单元、FU2 作为访存指令和特权指令的执行单元、FU3 作为乘除法指令的执行单元、BU 作为分支跳转指令的执行单元。指令执行完毕后，将数据写入指令 ROB 索引对应的位置，同时将对应该位置的 complete 信号置为 1，表示指令已经执行完毕。

2.4.6 重排序缓存模块设计（ROB）

该模块内部实例化一个 FIFO 作为存储部分，每一项储存一条指令退休时需要使用的数据，包括指令 pc 值、指令执行结果、访存地址、特权指令相关信息、分支预测信息和异常信息等。

在重命名阶段，指令读 ROB 返回指令将要放入的 ROB 索引，并在进入 ROB 时将对应该位的 complete 信号置 0，表示指令未执行完毕。

在发射阶段，指令根据源寄存器的映射关系读取 ROB 对应位的 complete 信号和数据。

在执行阶段结束时，指令将执行结果存入 ROB 对应位置处，并将对应位的 complete 信号置 1，表示指令执行完毕。

在提交阶段，ROB 将允许退休的指令数据送至提交阶段，将退休指令的目的寄存器和 ROB 索引送至 RAT 进行无效化，然后在下一时钟上升沿修改头指针，完成指令的出队。

当分支预测错误发生重定向时，将 ROB 尾指针修改为该分支预测指令的 ROB 索引处，实现分支预测错误后 ROB 的恢复工作。

2.4.7 提交模块设计（COMMIT）

提交模块每周期从 ROB 中取出两条指令，先根据指令信息判断是否进行双提

交：1.后一条指令被打上重取标记时；2.后一条指令为特权指令时；3.后一条指令为 ll.w 或 sc.w 指令时；4.后一条指令为分支跳转指令时；5.前一条指令发生例外时；6.前一条指令为 ertn 指令时；7.前一条指令被打上重取标记时；8.ROB 中仅有一条指令时，进行单提交，其他情况进行双提交。

当指令退休时，根据指令信息进行写寄存器堆、写 CSR 寄存器、写 TLB、处理例外等操作，若指令需要写寄存器堆，则还需对 RAT 进行无效化工作。

2.5 转接桥模块（AXI_bridge）

AXI_bridge 模块作为缓存系统与 AXI 总线之间的桥梁，负责处理指令缓存(ICache)和数据缓存(DCache)与 AXI 总线之间的所有读写操作。该模块通过状态机控制数据流，确保在满足 AXI 总线协议要求的同时，实现高效可靠的数据传输。

读请求状态机：

read_request_empty：空闲状态，等待读请求

read_request_ready：读地址已准备好

读响应状态机：

read_respond_empty：等待数据传输开始

read_respond_transfer：数据传输进行中

写请求状态机：

write_request_empty：空闲状态，等待写请求

write_data_wait：等待写地址握手

write_data_transform：数据传输状态

write_wait_b：等待写响应

2.6 cache 设计

2.6.1 icache 设计

本处理器的 icache 设计采用四路组相连阻塞式状态机设计，设计思路以及信号命名参考 LoongArch 版 CPU 设计与实战。参考存储采用 block memory 作为存储模块，icache 大小为 16KB，使用 VIPT 查询方式，主存块标记 tag 为 20 位，cache 索引地址 index 为 7 位，字块内偏移 offset 为 4 位，因此一个 cache 行可以存储 4 个字。Icache 替换策略为无效路优先的随机替换。

Icache 支持每次从 cache 中取出一行数据交给前端处理，如果不命中，则向总线发读请求。

单 cache 行查找的状态机转换：

IDLE: 初始状态，处理完取指请求后保持的状态。若发出取指令请求，同时没有 cacop 请求，进入 LOOPUP 查找状态，并在此时将查找 index 地址等送入 cache 行进行查找比对，以便能在下一个周期返回比对结果，否则保持在原状态。

LOOKUP: 查找阶段，根据上一个周期传入的地址得到的数据块判断是否命中，若命中，则直接返回一行 cacheline 数据，并检查 valid 是否置高，如果置高，则说明又有访存请求的发出，状态机仍为 lookup，如果没有 valid 的发出，则直接返回到 IDLE 状态。

如果 cacheline 没命中，状态机转换到 miss 阶段，进行后续的选择路替换。

Miss 阶段: 等待 wr_rdy 置高，同时根据替换策略选取替换路，等待下一个阶段发出，下一个阶段进入 replace 阶段。

Replace 阶段: 发出读请求，根据替换块是否为脏决定是否发出写请求。同时等待 rd_rdy 置高，如果 rd_rdy 置高时，状态机转换到 Refill 阶段。

Refill 阶段: 等待读总线返回数据，内部有一个 ret_cacheline 寄存器，用来存

值从总线返回的数据。在最后一个数据返回时置高 `data_ok`，状态机置回 IDLE。

2.6.2 dcache 设计

本处理器的 dcache 设计采用四路组相连阻塞式状态机设计，设计思路以及信号命名参考 LoongArch 版 CPU 设计与实战。参考存储采用 block memory 作为存储模块，dcache 大小为 16KB，主存块标记 tag 为 20 位，cache 索引地址 index 为 7 位，字块内偏移 offset 为 4 位，因此一个 cache 行可以存储 4 个字。dcache 替换策略为无效路优先的随机替换。

dcache 的状态机转换：

IDLE：初始状态，处理完取指请求后保持的状态。若发出取指令请求，同时没有 cacop 请求，进入 LOOPUP 查找状态，并在此时将查找 index 地址等送入 cache 行进行查找比对，以便能在下一个周期返回比对结果，否则保持在原状态。

LOOKUP：查找阶段，根据上一个周期传入的地址得到的数据块判断是否命中，若命中，则根据索引返回数据，并检查 valid 是否置高，如果置高，则说明又有访存请求的发出，状态机仍为 lookup，如果没有 valid 的发出，则直接返回到 IDLE 状态。

如果未命中，状态机转换到 miss 阶段，进行后续的选择路替换。

Miss 阶段：等待 `wr_rdy` 置高，同时根据替换策略选取替换路，等待下一个阶段发出，下一个阶段进入 replace 阶段。

Replace 阶段：发出读请求，根据替换块是否为脏决定是否发出写请求。同时等待 `rd_rdy` 置高，如果 `rd_rdy` 置高时，状态机转换到 Refill 阶段。

Refill 阶段：等待读总线返回数据，向 cacheline 中依次写入返回的数据，当所需数据返回时 `data_ok` 置高，当全部数据返回完毕，状态机返回到 IDLE 状态。

dcache 写状态机的转换：

IDLE：初始状态，当接受到写命中请求，状态机转换到 WRITE 阶段，并且

write_buffer 读入需要写入的数据。

WRITE 阶段：将数据写入 cache，同时将脏位置 1，如果此时仍然有 hit_write 信号，则保持在 WRITE 阶段。否则则回到初始状态。

2.7 地址翻译设计

2.7.1 Addr_trans 模块：

Addr_trans 模块是 tlb 的顶层模块，根据传入的 csr 信息判断是否采用 tlb 中的信息，生成最终的地址翻译结果，取指，访存模块共用。模块设计以及信号生成参考 openla

2.7.2 Tlb 模块：

Tlb 设计思路参考 CPU 设计与实战。Tlb 模块有两个查询端口，分别为指令地址和数据地址使用，查询当拍返回，支持指令数据并行查询。以及一个读端口，写端口以及一个 tlb 无效端口。读端口当拍返回，写端口同步写入。

2.8 SoC 设计

我们在 chiplab 提供的 SoC 的基础上，设计了我们自己的 SoC。我们的 SoC 包含以下外设：

- DDR3 DRAM
- NAND Flash
- SPI Flash
- UART 串口控制器
- Ethernet 控制器
- GPIO
- LCD 显示器
- VGA 输出设备
- PS2 输入设备

下图是我们的 Soc 架构图：

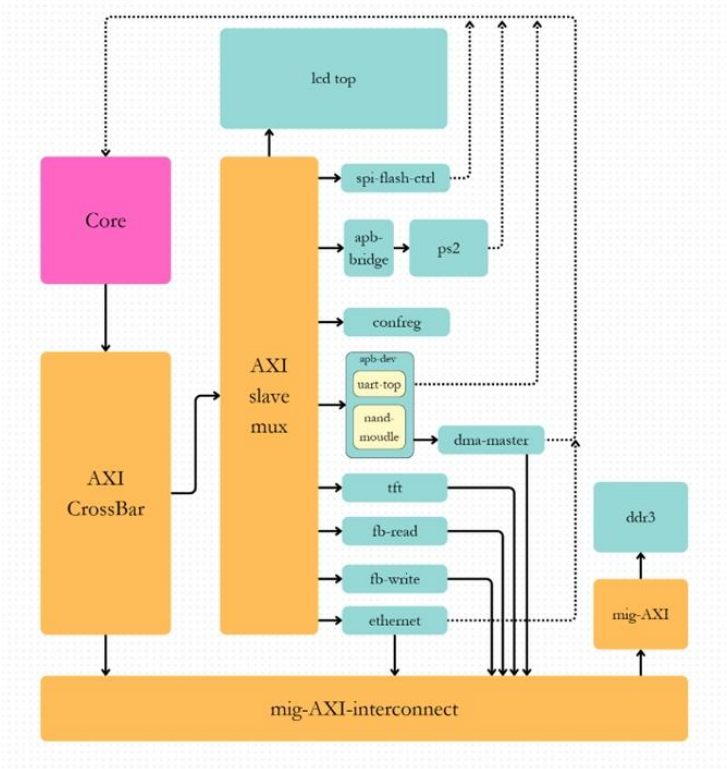


图 1 Soc 设计架构图

2.8.1 地址分配

SoC 对各外设的地址映射规则如下表所示。

表 1 地址映射表

地址段	起始地址	结束地址	分配大小
DDR3 Memory	0x00000000	0x07FFFFFF	128MB
SPI Memory	0x1C000000	0x1C0FFFFFF	1MB
TFT Controller	0x1D010000	0x1D01FFFF	64KB
PS2 Controller	0x1D020000	0x1D02FFFF	64KB
Video Frame Buffer Read Controller	0x1D050000	0x1D05FFFF	64KB
Video Frame Buffer Write Controller	0x1D060000	0x1D06FFFF	64KB
LCD Controller	0x1FC00000	0x1FC0FFFF	64KB
Configuration Registers	0x1FD00000	0x1FD0FFFF	64KB

UART Controller	0x1FE00000	0x1FE0FFFF	64KB
NAND Controller	0x1FE70000	0x1FE7FFFF	64KB
SPI Controller	0x1FE80000	0x1FE8FFFF	64KB
Ethernet Controller	0x1FF00000	0x1FF0FFFF	64KB

2.8.2 外设控制器

我们移植了一些已有的外设控制器，并且沿用并拓展了 chiplab 使用的外设中断号分配，详见下表。

表 2 控制器配置及来源

外设	控制器来源	中断位
Ethernet	Chiplab	HWI0
UART	Chiplab	HWI1
SPI Flash	Chiplab	HWI2
NAND Flash	Chiplab	HWI3
DMA	Chiplab	HWI4
PS2	iFu-Processor	HWI5
VGA TFT	Xilinx	-
Video Frame Buffer	Xilinx	-
Read		
Video Frame Buffer	Xilinx	-
Write		
LCD	在 404-NOT-found-Processor 的控制器基础上改进	-

2.9 VGA 控制器

2.9.1 VGA 控制器设计

本系统实现了一个基于异构计算架构的高性能、无撕裂 VGA 显示控制器。其核心是一个由多个专用硬件 IP 核构成的并行处理流水线，通过 AXI4 互联架构协同工作，实现了从内存到屏幕的端到端硬件加速数据通路。

运行阶段始于帧缓冲读取控制器的启动。该控制器作为主设备发起高效的 AXI4 突发读事务，以最大化内存带宽利用率的方式从 DDR 系统内存中获取原始图像数据。读取的数据被实时重组为符合 AXI4-Stream 协议的视频流，其中嵌入的 TUSER 信号标识帧起始，TLAST 信号标识行结束，从而维持了视频流的时空结构。

视频流随后进入流处理器进行实时处理。该处理单元在像素级流水线上执行可配置的图像处理算法，可能包括但不限于色彩空间转换、分辨率缩放、以及基于查找表的色彩校正。处理过程以线速进行，确保了恒定的处理延迟，从而满足了实时显示系统的时序要求。

经处理后的视频流被馈送入帧缓冲写入控制器。该控制器同样作为 AXI4 主设备，将处理后的像素数据以突发写事务形式写入指定的后台帧缓冲区。此过程严格遵循双缓冲机制：显示控制器（`axi_tft`）始终从前台缓冲区读取数据以驱动显示设备，而视频处理流水线则向后台缓冲区写入处理完毕的下一帧图像。这种隔离机制从根本上消除了读写竞争导致的屏幕撕裂现象。

缓冲区交换是系统的关键同步操作。当一整帧数据完全写入后台缓冲区后，系统通常会在垂直消隐期（**Vertical Blanking Interval**）——即前一帧显示结束、后一帧尚未开始的时序窗口——通过 CPU 干预或直接内存访问（DMA）触发一次原子性的指针更新。具体而言，CPU 将 TFT 控制器的帧缓冲区基地址寄存器重新映射到已完成写入的后台缓冲区地址。此后，原先的后台缓冲区晋升为新的前台缓冲区用于显示，而原先的前台缓冲区则变为新的后台缓冲区，接收后续处理流程的输出，从而完成一个完整的处理与显示周期。

最终，TFT 显示控制器独立工作。它基于预先配置的 VGA 时序参数（包括水

平/垂直同步脉冲宽度、前沿与后沿时间)生成精确的时序信号(HSYNC, VSYNC)。同时，它通过 AXI4 总线从当前的前台缓冲区中读取像素数据，并将其转换为模拟信号或低压差分信号以驱动物理显示设备。

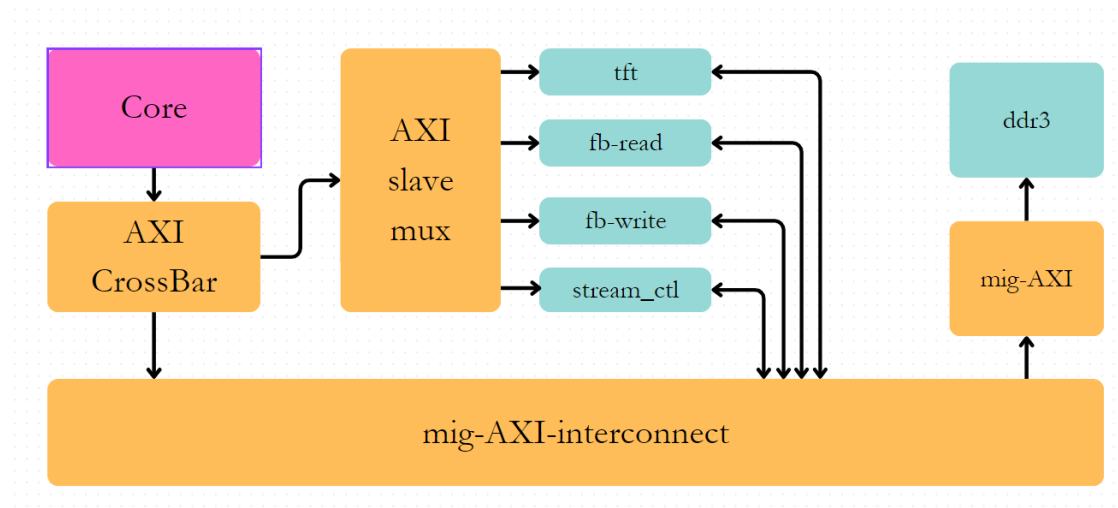


图 2 VGA 控制器架构图

2.9.2 VGA 控制器展示

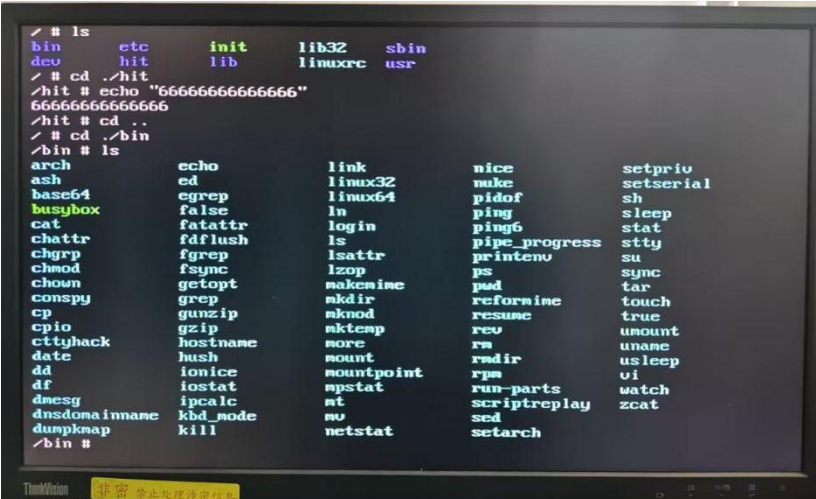


图 3 将输出重定向到显示器。

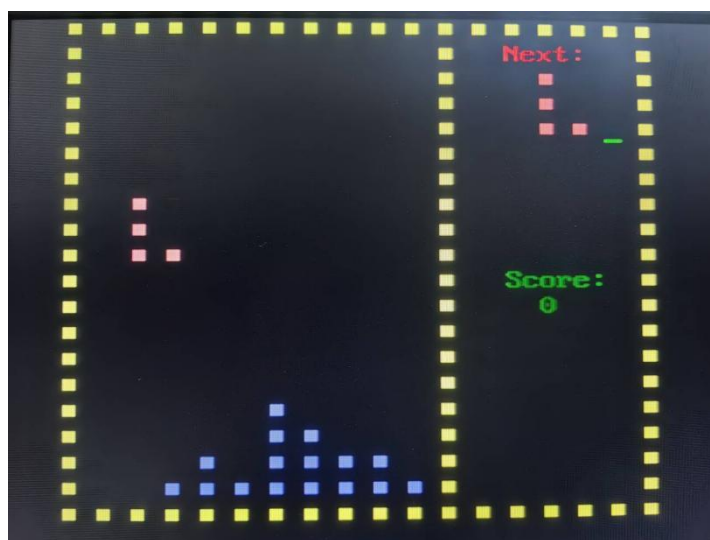


图 4 使用显示屏运行俄罗斯方块小游戏

2.10 LCD 控制器

2.10.1 LCD 控制器设计

本设计中的 LCD 控制器是一个功能完整、层次分明的系统模块，旨在实现 CPU 与 LCD 显示屏之间的高效通信与数据交互。整个控制器以 `lcd_core` 为核心，协调各个子模块的工作，确保显示内容能够准确、及时地呈现在屏幕上。控制器通过 AXI 总线与 CPU 进行通信，接收来自 CPU 的读写请求，并完成相应的数据传输与控制操作。其中，`lcd_ctrl` 作为控制逻辑的主要模块，负责解析 CPU 发出的 AXI 信号，并与系统总线完成握手协议，确保数据传输的稳定性与正确性。该模块内部集成了译码逻辑，能够将 AXI 协议中的地址、控制信号等信息翻译为 LCD 接口可识别的控制命令，并用于更新 `lcd_interface` 中寄存器的值。同时，`lcd_ctrl` 也承担着反向翻译的功能，将接口模块的状态和数据封装为符合 AXI 规范的信号，反馈给 CPU，从而实现双向通信。

`lcd_interface` 模块是控制器与 LCD 硬件之间的桥梁，其内部包含一组可供 CPU 和 LCD 直接访问的寄存器。CPU 通过标准的存储器写（`sw`）或加载（`lw`）指令对这些寄存器进行读写操作，从而下达显示命令或配置参数。该模块根据寄存器的内容以及来自控制逻辑的指令，向 LCD 发送具体的命令序列，如清屏、光标设置、

数据写入等。为了实现命令的准确传递，系统中还设计了 `lcd_id` 模块，其作用是对写入数据（`wdata`）进行解析，识别出具体的控制指令，并将其转发至 `lcd_interface`，确保命令的正确执行。此外，`lcd_init` 模块负责 LCD 的硬件初始化流程，在系统上电后自动执行一系列初始化序列，使 LCD 进入正常工作状态。该模块不仅完成了必要的硬件配置，还集成了开机启动画面，在初始化完成后自动显示“HIT”LOGO，提升系统的用户体验。

在多源控制信号的处理上，系统采用了 `lcd_mux` 选择器模块，用于在 `lcd_id` 和 `lcd_init` 两个信号源之间进行选择，实现初始化流程与用户命令之间的切换。当系统处于启动阶段时，`mux` 优先选择 `lcd_init` 输出的初始化指令；初始化完成后，则切换至 `lcd_id`，允许 CPU 通过寄存器写入控制 LCD 的后续操作。为了支持触摸功能，系统集成了 `lcd_touch_scanner` 模块，该模块参考了 UltraMips 平台的触屏扫描逻辑，具备即插即用的特性，可直接作为 IP 核集成到控制器中，实现对触摸事件的检测与上报。同时，为了提供更灵活的显示控制，设计中还包含了 `lcd_refresh` 模块，用于实现屏幕的刷新操作，能够清除当前显示内容，恢复白屏状态，为新的图像或文字绘制提供干净的画布。

在字符显示方面，系统引入了 `char_ctrl` 字符控制器与 `char_lib` 字库模块协同工作。`char_ctrl` 负责根据待显示字符的 ASCII 码，从 `char_lib` 中获取对应的 24×40 像素点阵数据，并按照扫描顺序逐点生成 RGB 颜色值，发送至 `lcd_ctrl` 进行显示。`char_lib` 则存储于 BRAM 中，覆盖 ASCII 码 32 至 128 范围内的常用字符，确保文本信息的完整呈现。`char_ctrl` 与 `lcd_ctrl` 之间设有握手机制，确保数据发送的时序正确，避免因速率不匹配导致的显示错误。整个 LCD 控制器以 `lcd_core` 为调度中心，实时监控系统状态，判断当前应显示的内容来源，是启动画面、用户文本还是图形界面，并据此协调各模块的运行，最终实现一个稳定、高效、功能丰富的 LCD 显示控制系统。

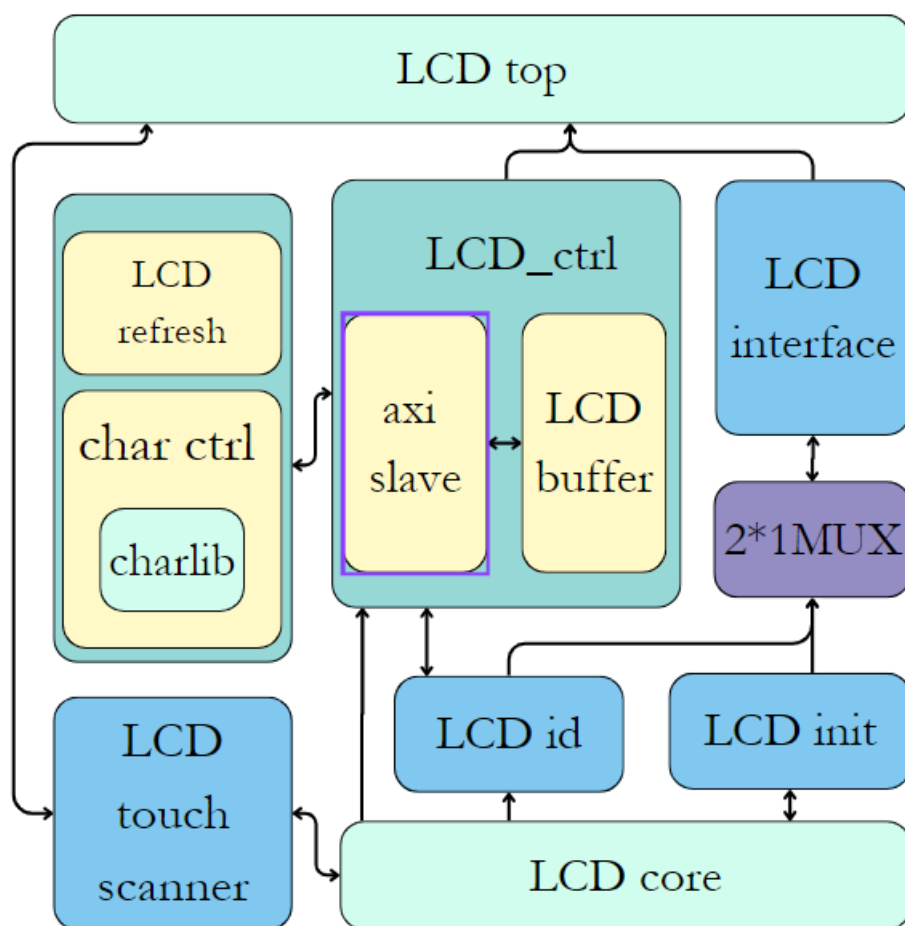


图 5 LCD 控制器架构图

2.10.1 LCD 控制器系统展示

除了作为系统输出设备，LCD 屏幕本身具备独立运行简单逻辑的能力（无需 CPU 持续干预）。可以运行随机数生成显示程序，动态更新屏幕数字。同时可执行色彩变换程序，更改主界面的颜色。



图6 LCD 控

制器

的独立运行功能，其中左上为主界面，右上为通过点击右下方的方块更改主界面颜色，左下为通过点击左下方方块进入的随机数生成模式，右下为点击左上方块将控制权转交给 CPU 后

的界面，点击右上角红色方块可以回到主界面

在启动系统后，我们写了一个贪吃蛇小游戏，在 LCD 屏幕上，流畅运行贪吃蛇小游戏。小游戏中应用了 lcd 屏幕的触屏功能，可以移动下面的滑块来设置游戏速度，并且点击 START 键开始游戏。触屏的反应，蛇身移动、食物生成和碰撞检测均表现流畅，操作无延迟，验证了系统在较小屏幕上的图形渲染效率和 I/O 性能。

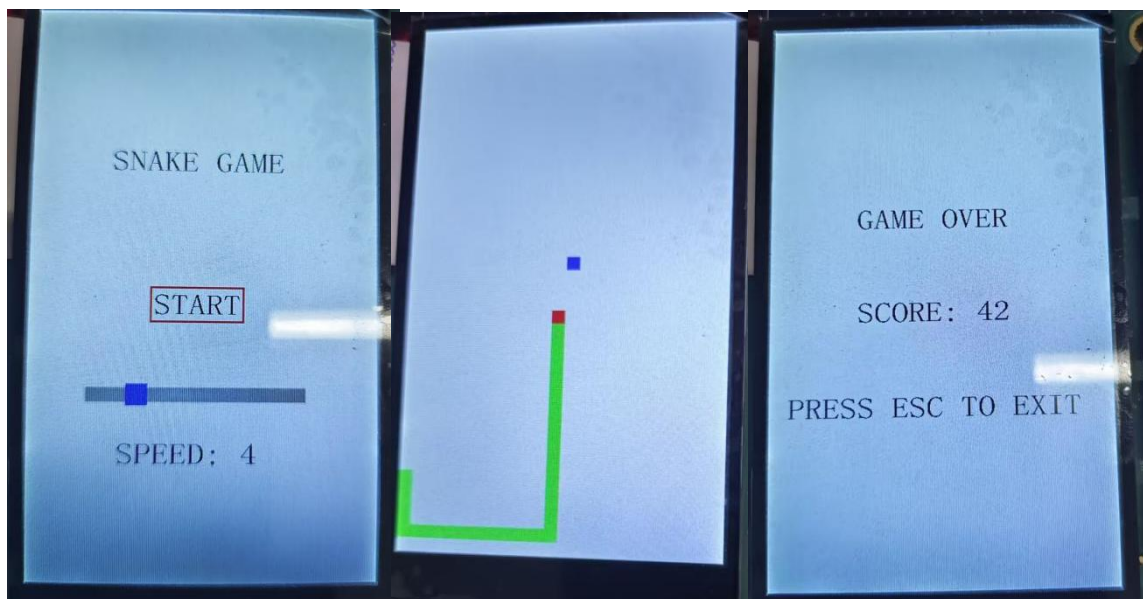


图 7 利用 lcd 外设开发的贪吃蛇小游戏，其中开始部分的速度设置和游戏开始使用了触屏功能

除此之外，该系统已实现将 shell 的完整 TTY 会话重定向到 LCD 屏幕上，可以正常运行指令，支持标准换行与刷新功能。当内容超出屏幕底部时，将自动上滚一行显示。

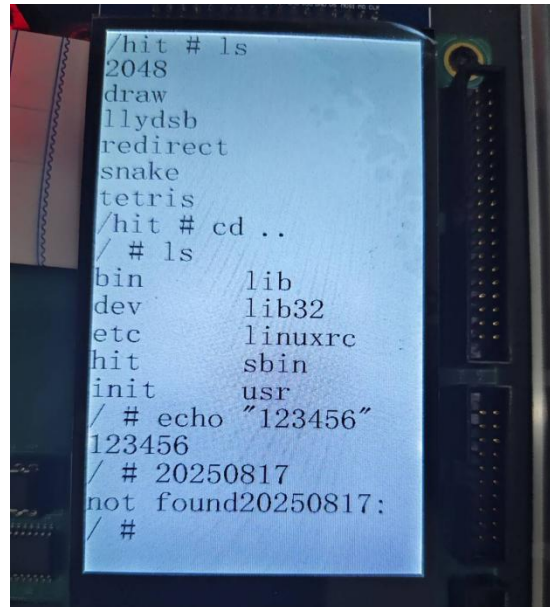
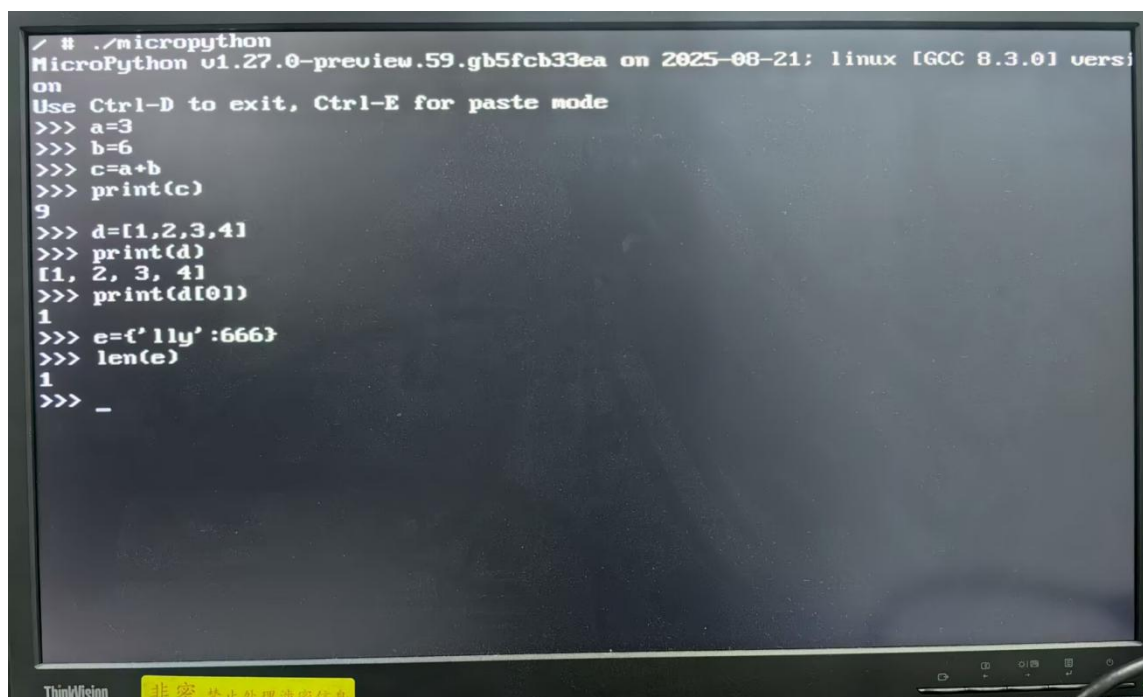


图 8 将 shell 会话输出重定向至 LCD 屏幕

2.11 micropython 移植

除此之外，我们还移植了 micropython 以验证我们的 CPU 的可靠性。



```

/ # ./micropython
MicroPython v1.27.0-preview.59.gb5fcb33ea on 2025-08-21; linux [GCC 8.3.0] vers
on
Use Ctrl-D to exit, Ctrl-E for paste mode
>>> a=3
>>> b=6
>>> c=a+b
>>> print(c)
9
>>> d=[1,2,3,4]
>>> print(d)
[1, 2, 3, 4]
>>> print(d[0])
1
>>> e={'lly':666}
>>> len(e)
1
>>> _

```

图 9 在 linux 上运行 micropython

第 3 章 关键技术及特色/亮点

所有的关键技术在上 one 章节已经详细介绍不再赘述。设计中的亮点主要有，前后端解耦合，取基本块结构前端，后端通过寄存器重命名来解决前递逻辑，已经初步具备乱序执行潜力，稳定的 LCD 控制器，能支持 VGA 和 PS/2 的 SoC。

第4章 调试及优化

4.1 测试方法

4.1.1 备赛初期

备赛初期采用《CPU 设计实战：LoongArch》提供的测试框架，该测试框架采用基于 vivado 仿真的 trace 比对的模式，通过捕捉和比较 CPU 对体系结构寄存器的写行为来判断 myCPU 的功能是否正确，上手比较快（测试框架详细使用说明见书本），完成设计实战所有测试后，CPU 的功能基本就完善了。

《实战》提供了总共 19 个测试，对应 CPU 功能实现顺序为：

5 条指令单周期 CPU→实现最基本的 20 条指令→将 CPU 改为简单五级流水
→所有 46 条用户态指令→支持精确异常（此时有 58 个测试点，同时也是初赛功能测试要求的测试点）→实现类 SRAM 总线（为流水线和 Cache 的交互模式奠定基础）→实现 AXI 总线（实现 AXI 总线后能通过初赛功能测试和性能测试）→实现地址翻译、TLB→实现 Cache 和 CACOP（此时共有 79 个测试点）

4.1.2 备赛中后期

参考《实战》完成其提供的 79 个功能测试点后，就可以着手修改 CPU 以 chiplab 测试框架。chiplab 的差分测试模型对处理器状态进行了非常详细的比对，需要从 CPU 内部提供较多的信号。改动 CPU 代码并且学习 chiplab 仿真的工作流需要大约 1~2 天，考虑到后期大部分的功能完善都要依托于 chiplab 提供的测试框架，因此这点时间花费是非常值得的。

适配 chiplab 测试框架之后，首先继续完善功能，chiplab 最后一个功能测试 func19 总共 81 个测试点，其中前面 79 个测试点和设计实战相同，多出的两个测试点是 sc.w 和 ll.w，为两个特权态指令，并且是启动系统所必需实现的指令。

完成 chiplab func19 之后，启动系统所需的最基本的指令和功能就都实现了（当

然 CPU 肯定还存在较多的缺陷），此时可以开始尝试系统启动，在报告的后面会提到系统启动的方式方法（详见 4.3 系统软件调试），这里不再赘述。

4.1.3 在线评测

NSCSCC2025 首次对团队赛采用了 gitlab CI 进行在线测试，测试的原理是将所有设计代码和 IP 核上传到远程仓库，服务器自动生成对应的 bit 流产物，将产物烧录到远程 FPGA 得出结果。

按照规定，所有的成绩认定应当都以在线评测的结果为准。官方大概率会在报名成功之后的一个月内逐步发放在线评测账号和仓库，并且确定明年起会将各个队伍的性能测试结果输出到多维表格中（可能面向所有参赛队伍公开，更加透明但是竞争压力会更大）。

但是要注意在线评测的结果具有不稳定性，今年通知要采用在线评测时距离复赛提交只剩两天。我们在这两天内尝试了各种版本和频率，只有一个超频的版本能在线通过所有测试（因为时序违约所以不能作为提交成绩），甚至初赛提交的版本在远程一个测试都不能通过。但是我们将 CI 生成的 bit 流下载，烧到自己的板子上却能正常通过所有测试，说明产物本身没有问题，只是在线评测烧板出现了某种异常，这个问题直到复赛提交截止时还没有修复，很多参赛队也反映有同一套代码同一个频率出现了重复提交不能稳定通过所有测试的情况，后面和主办方协商的结果是一定要采用线上生成的产物进行测试，自己提交的分数成绩可以使用本地手测的结果，官方会使用同一套产物进行手动复测。

目前不知道官方是否已经对在线评测的技术流程进行了修复，但是有一点是可以肯定的，在线评测是团队赛大势所趋。

4.2 调试和优化过程

因为我们将比较多的精力花在启动系统上，所以并没有特别多的精力对 CPU 的性能进行优化，也没来得及建立比较完整的工作流来衡量 CPU 的性能水平。

初赛到复赛这段时间，因此要保证系统功能，所以也不能对 CPU 架构进行太激进的改变，我们主要通过扩大缓存容量、优化调用返回栈、打开后端前递等简单

的手段进行优化（虽然简单但是效果很好，尤其是缓存容量扩大带来的提升非常大）。总体来说复赛提交版本相比初赛版本性能提升 50%左右，与其说这些是优化手段，倒不如说是我们一开始在设计上产生了一些偏差导致初赛提交版本性能严重不达预期。

我们对 CPU 性能进行衡量主要选取了三个指标：前端分支预测准确率，后端双发率，缓存命中率。

在性能优化的过程中，主要通过修改官方提供的 vivado 工程中的 myCPU_tb 脚本，在性能测试仿真过程中统计各项关键数据，并且在仿真结束之后打印到控制台上。

由于 vivado 仿真速度较慢，对于体量比较大的测试仿真时间成本会非常高（我们使用 i9-13980HX 并进行超频，跑单次 fireye_A0 测试需要将近两个小时时间），因此我们一般跑 bitcount 测试来衡量流水线效率，跑 stream_copy 测试来衡量访存系统效率。

在 linux 环境下使用 verilator 进行不记录波形的仿真速度肯定会快很多，但是由于官方今年新增了 CPUCFG 指令来获取 Cache 配置（每个测试点开始运行前要进行 Cache 初始化），而 verilator 所依赖的差分测试模型还不支持 CPUCFG 指令，所以我们并没有使用 verilator 来进行性能测试仿真（实际上应该可以关掉差分测试模型直接跑仿真，但是我们没有尝试）。如果要在性能优化上投入较多的精力以达到更优的性能，那么应当首选 verilator 进行仿真，只有在测试点出现不能通过的情况下才使用 vivado 进行有波形的仿真。

4.3 系统软件调试

4.3.1 调试中遇到的问题和解决办法

通过 chiplab func19 的所有功能测试后，我们着手进行系统启动过程的仿真和尝试上板 boot。过程中遇到的问题实在太多了，这里只讲讲启动过程中被哪几个大问题卡住以及最后是怎么解决的。

一开始我们采用 PMON 引导的方式，PMON 启动过程会输出提示信息，

我们发现总在输出“BEV0”这个地方卡住，进入不了 PMON 提示符，并且那个时候我们还不知道 chiplab 能对 PMON 启动过程进行仿真（实际上只要有能编译的 C 代码或者可执行二进制文件都可以进行仿真）。

于是问题就变得棘手起来，我们根本不知道该如何检查出 bug，然后了解到可以通过 vivado 的 ila 抓波形，但是抓波形的前提是想要抓的信号没有在综合过程中被 vivado 优化掉，并且针对关键信号设置一个正确的触发条件，才能抓取触发条件满足时，各个信号在附近一段时间内的波形。

一开始我们缺乏抓波形的经验，对想要的信号在代码中修饰了 `mark debug = TRUE` 却还是发现信号被 vivado 优化掉（后来才知道还要加上 `keep=TRUE`，vivado 才不会动那个信号）。而且就算我们偶尔运气好，有一些信号想要的信号没有被优化掉，我们也不知道该设置什么触发条件（因为根本不知道问题在哪里）。于是我们陷入了好几天的迷茫期，从遇到这个问题开始总共花了 4 天时间尝试探索抓波形这个解决办法，但是最后还是看不到一点解决希望……

最后是我们终于发现，利用官方给出的 PMON 二进制程序，可以在 chiplab 中进行仿真，我们才算是找到了一点解决希望。可是仿真过程中出现报错的位置并不是输出“BEV0”的位置（这是一个疑点，但是我们没有深入进行分析，总之出现问题就修复呗）。发现问题和 `uncache` 信号有效的时候，`Dcache` 的访存行为不正确有关（运行过程中输出字符涉及到和 `uart` 模块的通信，属于 `uncache` 访问）。修复了这个问题之后上板竟然能够跑过“BEV0”这个位置，并且成功进入 PMON 提示符。

但是我们发现进入 PMON 提示符之后，键盘输入时串口并不会立即响应，呈现出一种奇怪的规律，每当我们输入 15 个字符之后再继续输入，先前输入的字符才会一个个依次在命令行中吐出来。我们仔细阅读了 `uart` 模块的代码，发现其中有一个大小为 16 的 FIFO 队列用作缓冲，当这个缓冲区满时，`uart` 模块会向 CPU 爆出硬件异常，让 CPU 读取 FIFO 队头的一个字符。

当时我们的推断是，`uart` 向 CPU 爆出硬件异常时，CPU 肯定会进入异常处理程序，并且在异常处理程序中肯定执行了从 `uart` 中读出一个字节数据（ASCII 码

一个字符的大小）的指令。既然爆出异常的时候我们的 CPU 能正常读出，那么是不是说明实际上 CPU 从 `uart` 中读取数据的功能是没有问题的呢？（看上去很合理的推断，可惜后面发现是非常错误的），并且我们的 CPU 能通过进入异常处理程序的方式读取字符，至少说明 CPU 当时并没有卡死，此时这个 bug 又显得非常诡异了，看起来哪里都没问题……

那么上板出现问题，首先会想到的当然是能不能跑仿真来检查。但是很遗憾，我们尝试对 PMON 运行全过程进行仿真，由于 PMON 启动过程会扫描 NAND 设备，我们一开始只对 `chiplab` 的测试框架进行了简单更改，似乎并不能支持在仿真过程中对 NAND 设备进行模拟（官方提供了 `linux` 仿真，而 `linux` 启动过程也会扫描 NAND，所以 NAND 行为应该是可以仿真模拟的，只是我们不知道如何更改），于是每当 PMON 仿真中输出完“`scanning NAND`”时，就会进入死循环，我们一开始还以为是执行过程比较长，后来才通过比较复杂的办法确定仿真确实是在死循环，也就是说运行 PMON 仿真并不能进入 PMON 提示符，所以我们遇到的 bug 不能通过仿真来解决了……

那么最后这个问题是怎么得到解决的呢，是莫名其妙解决的，没错！就是莫名其妙地被解决的，我们那时上板抓波形也不知从何入手，只能先跑点别的仿真测试看看，然后我们碰巧运行了 `chiplab` 中自带的“`hello world`”测试（这个测试涉及到非常多的 CPU 和 `uart` 交互），然后发现仿真报错了！对“`hello world`”运行中出现的 bug 进行修复，我们发现是一个非常隐蔽的问题：之前对访存指令进行设计的时候，我们模仿 `openla500` 中的 `mem_stage` 模块中的写法，直接尝试从地址对齐处读取 32 位的数据，然后对返回的数据进行裁切。我们的 Dcache 面向总线发起访问请求也沿用了这种思路，如果当前 Dcache 收到 `uncache` 信号有效（如果没有 `uncache` 信号那么直接从总线读取一个缓存行了，这是一般的访存行为，访问外设时一定是 `uncache` 信号有效的），那么 Dcache 向 AXI 总线发起请求的数据宽度（由 `arlen` 信号控制）设定为 32 位，Dcache 从总线中读取 32 位数据后直接返回给流水线。这样实现看起来非常地合理，但是“`hello world`”仿真测试的 bug 表明，向 `uart` 读取数据的时候，`uart` 模块只支持一次从它的 FIFO 队头位置读取一个字符

（也就是 1B），实际上流水线内部的指令也是 ld.b，只是我们的访存行为实际上相当于执行 ld.w，也就是向 uart 模块一次请求 32 位的数据，此时 uart 并不能返回队首的数据，它内部的 FIFO 队列也不会有任何的变化，于是就表现为一开始键入的时候命令行一直没有反应（但是为什么爆出异常之后 CPU 反而能读取了，我们并没有研究明白）。

搞清楚这个问题后，在 uncache 访问时 Dcache 根据要读取的数据宽度调整向 AXI 发送的 arlen 信号，终于能正常在 PMON 正常输入命令了，并且我们觉得 CPU 访问外设寄存器的行为基本上不会再出现问题了。

PMON 能正常输入命令后，下一步就是通过网口加载 linux 内核数据，这一步首先要配置 IP，然后果不其然又又又出现了问题。正常情况下执行完配置 IP 的命令后，PMON 提示符会重新弹出，但是我们的 CPU 执行配置 IP 命令时直接卡死了，我们又一次进入了束手无策的状态……

好在有了先前的经验，我们逐步意识到 CPU 的访存系统存在很多缺陷（前两个大问题都和访存有关）。我们首先假设问题存在在 Cache 中，于是我们通过将 uncache 信号一直拉高的方式来禁用缓存，相当于让 CPU 所有的访存行为都直接绕过缓存直接从总线中读取。这样如果 CPU 的流水线执行和总线行为是正确的话，理论上应当能稳定运行。

按照这个思路我们进行了上板测试，发现果然成功执行了配置 IP（但是在启动内核时遇到了 panic，也就是流水线仍然存在问题），那么当前的 bug 一定是由 Cache 引起的了，不过到这一步我们还是没能直接得到与问题相关的波形来进行修复，一方面我们让负责 Cache 的同学自查和反思，另一方面我们继续进行其他程序的仿真，总之不到万不得已不上板抓波形。

在 chipiab 中对 Uboot 进行仿真时（用和 PMON 一样的方法进行仿真），我们发现了一个 cacop 处理的错误。我们发现在运行 Uboot 过程中有 cacop 指令，而 PMON 启动的过程中没有，因此即使我们的 Cache 执行 cacop 指令时会出错，也不妨碍进入 PMON 命令行。修复完 cacop 问题后，我们竟然能成功配置 IP 了，也就是 PMON 在执行配置 IP 命令时有 cacop 指令。

完成配置 IP，然后通过网口将内核下载进 FPGA 上的内存，这个过程很顺利，接下来就是运行内核启动 linux。我们首先尝试进行仿真，chiplab 提供了 linux 启动过程的仿真，使用的是官方提供的 vmlinux（是一个非常精简的 linux 内核），即使这个内核被精简过，整个启动过程在仿真中还是需要运行约 3.8 亿条指令。此时我们还没有了解过 lightsss 组件，运行仿真时打开波形记录，将整个过程的所有波形全部记录，这导致仿真的速度及其慢。我们使用 openla500 运行过 linux 启动过程的仿真，全部记录波形的情况下需要至少一晚上的时间，产生至少 300g 的波形才能在仿真中进入 shell。

因此我们一开始调试 linux 内核的过程非常低效，经常是跑一晚上波形，第二天早上起来查看波形，花费数个小时定位问题的位置（波形的时间跨度非常大）并且修复问题。修复完问题之后我们会进行一次不记录波形的仿真，通常能在一个小时之内运行到出现报错的位置，但是一次修复往往不能成功（有时甚至会产生新的问题，将出错的位置提前），就算修复成功也会在后面遇到新的 bug，总之一天的修复结束之后，凌晨离开实验室时重新运行仿真记录波形，第二天继续修复。

具体在 linux 启动过程中遇到哪些问题我们已经记不清了，太多太多了，总之在尝试启动 linux 的第一周每天就是如此循环往复，低效且枯燥，修复完一个 bug 不知道后面还有多少 bug 等着。直到我们了解到 lightsss 工具后工作效率才提高了许多，lightsss 的原理是通过 fork 子进程，在差分测试模型报错的位置附近记录波形，而不是将所有波形都记录下来。因为使用 lightsss 避免了大量波形文件的写入，仿真速度相较全部记录波形快了非常多（还是比不记录波形慢一点），这样使得我们在修复问题后能立马跑仿真验证，只需要一个半小时就能验证，即使没有成功修复也能拿到波形重新分析。尤其是在进入 shell 的最后一两千万条指令，我们大概遇到了三个 bug，如果采用一般的全部记录波形的办法至少需要四天才能完成修复（毕竟出问题的点非常靠后，全记波形跑仿真肯定要很长时间），在采用 lightsss 之后我们在两天之内就解决了。

最后，当我们成功进入 shell 时已经是 7 月 23 号了，距离初赛提交只剩下 10 天左右，而此时我们只是使用简单五级流水 CPU 启动，而新架构才刚刚运行起

来……

4.3.2 调试工具

绝大多数是关于 `chiplab` 调试工具的用法在官方的用户手册上都说的很明白了，下面只说我们自己总结或摸索出来的一些用户手册上没讲的功能。

4.3.2.1 `chiplab` 仿真概述

`chiplab` 自身提供了差分测试仿真，使用方法见 `chiplab` 官方的用户手册。此外，我们还可以通过修改 `/chiplab/sims/verilator/testbench/difftest.cpp` 来实现一些自己希望额外实现的功能，比如每隔 10000 条指令在中断输出一次当前的 PC 值，以及设置 `csr` 寄存器的 `difftest` 的掩码等，这些都需要对 `difftest.cpp` 的源码进行一些小修改，具体修改你们可以自己研究一下，`difftest.cpp` 的源码不是很难理解（大不了问 AI），可以自己实现更多方便或者和自己的 CPU 更适配的功能。

4.3.2.2 `chiplab` Linux 仿真

具体看 `chiplab` 官方的用户手册。

4.3.2.3 `chiplab` PMON、U-Boot 仿真

PMON 和 U-Boot 仿真目前 `chiplab` 官方并没有提供直接的 PMON 和 U-Boot 仿真，但是可以利用 `chiplab` 提供的环境自己实现 PMON 和 U-Boot 仿真，甚至理论上，你只要有程序的二进制文件，所有程序都可以跑差分测试仿真。以我们比赛的时候的 `chiplab` 版本为例，在 `./software/bsp/env` 文件夹下有一个 `convert.c` 文件，就是起到了把 `.bin` 文件转换为 `rom.vlog` 文件的作用

具体来说步骤如下

1、	把 <code>convert.c</code> 编译成可执行文件 <code>convert</code>
2、	<code>./convert</code> <code>.bin</code> 文件的路径 生成的 <code>rom.vlog</code> 等文件的路径 如果出现了报错，那么大概是路径少加了/等

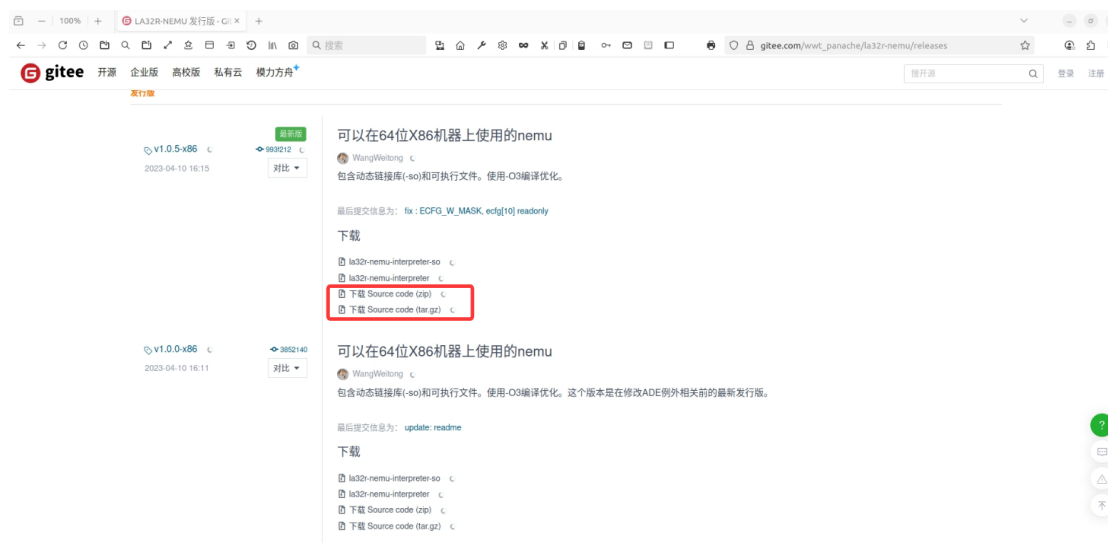
	比如说，如果我把 uboot.bin 放在了 ./software/bsp/env 文件夹中，然后希望把生成的文件放在 ./software/bsp/env/uboot，那么应该输入 ./convert u-boot.bin ./uboot/
3、	在 /chiplab/sims/verilator/run_prog 下跑任意一个仿真，比如 Linux 仿真，开始跑后 ^C 中断掉
4、	将生成的 rom.vlog 文件放置到 /chiplab/sims/verilator/run_prog/obj 下的对应的一个 _obj 文件夹下，比如第三步跑 Linux 仿真就在 linux_obj 文件夹中，替换原有的 rom.vlog。
5、	在跑仿真的终端运行 make simulation_run_prog，即可运行 uboot 或 pmon 仿真

4.3.2.4 chiplab 提供的 lightsss 仿真

关于 lightsss 的重要性前文已经提到过，总之进入到调试 linux 的阶段时一定要使用 lightsss。lightsss 的原理在官方文档已经写得很清楚了，这里不再赘述。

4.3.2.5 nemu 修改 tlb 条目

chiplab 官方提供的 nemu 二进制文件的 tlb 条目是 32，但是实际上有可能自己 CPU 的 tlb 条目不是 32，因此需要自己重新编译 nemu，下载下面的红色框的文件夹自己参考 readme 修改配置重新编译一遍就好了。



我们复赛提交版本采用了 8 条目 TLB，此时能正常启动系统，只是设计实战和 chiplab 的功能测试已经不能全部通过了，因为它们在测试过程中固定会访问超过索引超过 8 的 TLB 条目，此时会导致 CPU 行为的不确定，完全无法进行调试，建议在 CPU 其他功能都已经验证完备之后再着手减少 TLB 条目的个数。

第5章 关键问题及解决方案

我们遇到的关键问题基本上是系统启动过程遇到的问题，这部分已经在前面详细说明，接下来我们只讲一下在设计调试过程中积累的经验和运用的思想。

首先下一个定论，调试过程中遇到的 90% 的问题都必需依赖仿真进行解决，如果没有仿真，那么解决问题将变得非常麻烦棘手，所以遇到问题还是优先想想能不能通过仿真测试解决，即使没有直接的仿真手段，也要思考能不能找到相关问题的仿真测试来间接检验。

CPU 上板后的情况非常复杂，上板后的问题不一定都能找到对应的仿真测试（例如前面提到的 PMON 启动过程中遇到的问题），此时该怎么办呢？这时候就要学会从一个高层次去看待问题，逐步缩小问题的范围，然后对怀疑出问题的模块和功能进行不同场景下的针对性仿真测试，可能会找到解决问题的办法。

拿我们系统调试的过程来举例子，在调试过程中我们逐步明白 CPU 可以大体上分为两个模型：流水执行模型+访存模型

流水线执行模型指的是从前端取指令到指令被分发到后端执行，最后在后端执行完毕进行提交。流水线执行模型中需要做到正确处理以下几种情况：

pc 值没有产生异常时前端能否正确将指令取回。

前后端在出现分支重定向时能否正确进行冲刷，并且保证不会将错误的指令进行提交。

后端执行过程能保证指令得到正确的操作数（主要与后端前递逻辑有关），完整实现指令的功能（尤其特权态指令的正确实现）。

CPU 执行过程中产生的异常指的是在正确的指令流中的异常，如果一个异常本身属于错误指令流（会被分支重定向修正），那么不需要处理这个异常。

.....

对于访存模型，由于比赛要求实现 Cache 和 AXI 总线，同时启动系统又一定要实现地址翻译和 TLB，因此访存模型虽然功能单一，却是一个非常复杂的系统，

如果前期设计产生疏忽，很难保证在实际的复杂访存行为下总是正确的。访存模型主要在于做到以下几点：

给出正确的地址，能否将指令/数据取回。

出现访存相关异常（地址异常，tlb 异常），能否正确处理异常。

如果访存时禁用缓存，那么 CPU 内部相应的模块（可能由 Cache 一并实现）能否正确绕过缓存取回数据。

向 AXI 总线发出的请求和进行的处理是否正确（对于外设来说非常重要）。

如何保证缓存 CACOP 处理的正确性。

.....

在 CPU 设计过程中，我们首先采用面向测试用例的敏捷开发模式，分不同阶段逐步实现指令，通过简单测试用例初步验证功能是否正确，但是此时进行的测试是不全面的，甚至可以说是漏洞百出的。在实现指令的过程中不能认为跑通功能测试就万事大吉了，但是在这个阶段 CPU 功能还非常羸弱，指令没做完大部分仿真测试都不能跑，因此也只能依赖功能测试硬着头皮往下做，这个过程中缺陷会一点点积累，直到在调试系统的过程中集中爆发。

说回前面提到的流水线执行和访存模型，在 CPU 设计的过程中，由于每个阶段都要运行功能测试，而无论什么测试都要求流水线功能正常，能保证各种情况下指令正常执行（要不然根本就跑不起来嘛）。尤其是当 CPU 能顺利通过官方提供的性能测试时，此时我们认为流水线内部其实具备了较强的鲁棒性，对于正常情况下执行用户态指令一般不会出现错误。如果是流水线相关缺陷，大概率可能在处理异常时会出现问题（因为对异常处理的测试比较少）。异常处理相关的问题一般修复起来比较简单，甚至大部分的异常处理错误来源于对指令手册、参考核等相关资料理解不到位。

对于访存模型，由于加入 Cache 和地址翻译之后整个访存系统的复杂度非常高。《实战》中引入 Cache 之后，虽然对 cacop 指令设置了专门的测试点，但是总得来说测试数量还是太少，实际上板遇到 cacop 指令还是很容易出现错误。而对于 uncache 访问，《实战》压根就没有提供专门的测试来进行，而 uncache 访问往往

和外设相关，而外设功能是否正常在上板之后其实非常关键。总之，通过《实战》和 `chiplab func` 完成指令功能后，其实访存模型存在缺陷的可能性是很大的。我们在启动系统的整个过程中，大概 75% 的问题都和访存有关，而且一旦问题和访存相关，就变得极难排查（从前文就可以看出来，非常地困难且煎熬）。

我们后期在调试系统过程中遇到问题，通常采取的办法是：把 `uncache` 信号拉高，先让 CPU 禁用 `cache` 跑一下。前面已经提到过流水线执行已经很鲁棒，我们一般不优先考虑流水线出现问题，把 `uncache` 一直拉高然后跑各种仿真测试大概率能排查出 `uncache` 访存的缺陷并且修复。那么此时 CPU 相当于抛弃 `cache` 在“裸奔”，我们应当对此种情况下 CPU 的稳定性具有信心。

如果禁用 `cache` 后能通过出问题的地方，那么就可以把问题定位到 `cache` 内部，如果在同一个位置出错，此时优先考虑总线行为不正确，然后再考虑流水线的异常处理有缺陷，最后才考虑用户态指令执行过程中出现问题（这是在没有波形的情况下进行的推测，如果有波形直接分析修复即可）。

通过这个办法能大致对缺陷进行定位，如果用同样的思想继续对 CPU 根据功能进行划分，还能进一步将流水线分割为前端和后端分别看待，总之通过假设加上验证的办法一步一步将出现问题的范围缩小，直到没办法将问题的范围缩的更小就可以开始全面筛查。这就好像对问题进行“降维打击”，如果不采用这种办法尝试直接修复问题，免不了越来越狭隘，不仅可能被一个缺陷困住好几天，而且还可能耽误了其他功能的测试完善。

第 6 章 总结与展望

6.1 竞赛总结

一开始决定打龙芯杯的时候是大二秋季学期的 12 月，按照学校的课程安排，此时我们才上完数字逻辑，堪堪算是上手了 verilog 或者说数字电路设计，而对于计算机组成和 CPU 微架构是根本一窍不通的。

所以当初促使我下决定的初心是什么？好像已经模糊不清了，可能是因为自己有些清高，不想随大流去做 AI？又或者是觉得自己设计 CPU 是一件非常酷的事情，还有可能是对自己的能力很有自信，觉得只要花时间下去拿个一等奖特等奖不是问题（是的我们当初就是瞄着拿大奖去的哈哈）。

总之当初抱定了极大的决心，一路上也非常多的坎坷。寒假的时候我们四个人一起学计组，在 B 站上看刘宏伟老师的网课，后面才反应过来刘老师的计组课太抽象了并且很多内容都很过时，不过好歹是懂一点 CPU 微结构了。学完计组寒假差不多结束了，春季学期就跟着《实战》这本书慢慢做，一开始的时候我们还没有形成分工，每个人都会认真去做《实战》的实验，但是后面到学期中间的时候，大家伙都有事情要忙，队伍每一个人都做一遍《实战》的实验其实不现实，所以从精确异常那一块开始我们就慢慢分工了。分工之后我们还是做得很慢，因为课业压力真的非常大，我们队的每个人都是要努力保研的，每一门课都不能松懈，所以除了刚开学的那段时间进展比较快（主要做了流水线和用户态指令，大概 4 月初就完成了），后面一直到 5 月底，我们都处于一种，每周开会都想着把《实战》的实验早点做完（那会我甚至安排过一周做三个实验，实际上根本不可能，一周做一个就很不错了哈哈），但是一周过去预期目标远远没有达成，只能不断往后面安排。一直到 6 月要准备期末考试了，我们还是没有完全把 Cache 接好，也就是《实战》还有两个实验没有完成，没办法只能先准备考试。

比较好笑的一点是，我们一开始心比天高，但是在备赛的过程中一点点变得现实。自我评价一下，一开始我们确实有点无知无畏，B 站上有去年一等和特等的系

统展示视频，但是我们当初什么都不懂啊，也根本不知道他们做出的是怎样一种水平，后来在慢慢学会体系结构的知识后终于看明白了，知道自己根本不可能在几个月内做出这么优秀的作品。

但是等反应过来不对劲的时候，已经准备了好几个月了，总不能中途放弃吧，那就只能降低预期，朝着二等奖去努力（顺带一提，龙芯杯一等奖只发两个，太抠搜了，隔壁两个赛道都发 8 个一等）。

期末考试结束之后，我们只有两个月的时间，这时候我们还是只有一个功能不完善的五级流水 CPU，超标量和系统启动连影儿都没见着，不过那时候我们还是挺有锐气的，花了一周的时间就通过所有功能测试，然后我们兵分两路，一边改架构，一边用五级流水 CPU 尝试启动系统（后面发现这个思路是很正确的）。

我们的前端借鉴了 2022 年满洋学长的设计，满洋学长对于体系结构尤其是分支预测技术非常在行（后面他去计算所从事相关研究了），整个 7 月我都在仔细研究他的前端代码，包括前后端接口和前端内部时序，并且对设计的一些不足之处做了一些改进。总的来说，对于以后的参赛队伍，我都非常推荐参考满洋学长的前端，这个前端架构对于龙芯杯来说非常先进，并且还有很多潜力可以挖掘，一句话来评价：够我们学校的队伍接着用十年都不成问题。在此基础上我们的改动是根据自己设计的 Icache 重构了 IFU 的取指流水线，同时根据性能测试中调用返回的规律调整了 RAS 的设计，使得 RAS 的训练提前，并且相应修改了预译码器的设计。

当然我们还有不足之处，由于 Icache 不是流水线式的，因此在一些特殊情况下取指效率会非常低下，我们本来设想是 Icache 一周期返回一个基本块，这样能保证每周期最差也能取到一条指令。但是由于用的是状态机式的 Icache，因此实际上是两周期返回一个基本块，在遇到一些特殊循环时取指效率极低（主要和跨行截断的位置有关）。以及我在设计报告中提到使用 uBTB 来弥补前端更新不及时漏掉预测的问题（满洋学长的前端实际上没有使用 uBTB），但是我做出 uBTB 的时候已经临近决赛提交，因为引入 uBTB 导致前端出现新的缺陷，已经来不及修复了，所以复赛没有提交带有 uBTB 的版本。

对于后端，我们采用了动态调度的技术，并且使用了 ROB 来解决动态调度下

的前递逻辑，实际上做出 ROB 之后离乱序发射已经不远了，只需要更改唤醒逻辑以及做好恢复就行，但是我们已经没有足够的时间来调乱序了。7 月初还是简单五级流水，花了大半个月的时间就能做出动态我感觉已经是奇迹了，这里要感谢负责后端的队友，他真的太给力了。

总体来说，我们的架构在性能方便还是差强人意的，应该还存在一些较大的问题，尽管从初赛到复赛我们已经修复了很多对性能影响非常严重的问题了，不过毕竟我们没有多少时间来优化性能，对于二等奖来说，保证系统才是第一位的，虽然有遗憾但是还能接受。

调系统这里就不再赘述，前面已经详细讲述了过程，总之就是非常地煎熬且漫长，在这方面我们给出的经验应该是这几年参赛队中最多的了，前文详细讲述的各个缺陷其实也是大部分队伍会遇到的，以及问题定位的方法思想也讲了个大概。22 年的老前辈已经成为传说，23 年的队伍是通过改内核的方式来启动，说实话有点歪门邪道，24 年根本连启系统的希望都没看到，往后两年的队伍如果想在启动系统方面获得建议，可以再来咨询一下我们。

龙芯杯总体来说是难度非常大的比赛，尤其是在调试上，让你看一两天波形你可能还觉得乐在其中，让你看两个月波形你指定受不了，说实话我到 8 月初赛提交之后已经到了看见波形就想吐的程度了，同时比赛拿奖也非常难，因为一等奖发的少，传统强队积淀深厚，白手起家很难赶超，一个项目忙活大半年最后是二等奖其实心里会有些不好受的。但是如果你真的想在计算机体系结构，尤其是通用处理器设计方面有所建树，那么打这个比赛是非常好的锻炼，本科阶段没有比龙芯杯更加方向对口的比赛了，之后如果从事相关研究，做体系结构的老师们知道你做了一个功能完备，具备超标量技术能启动 linux 的 CPU，一定会对你很感兴趣。

这里再引用 23 年 Invalid Syntax 队伍报告中的一段话：

“当处在每个煎熬阶段的时候都感觉陷进了无底深渊，总是希望修完下一个缺陷之后就“柳暗花明又一村”；但是事实上，往往在已经被磨到没脾气的时候，已经麻木了的时候才能到达这个阶段的尽头，而在那时候已经不在能感受到成功是开心的了——因为已经麻木了。”

T2 6 楼，往主楼方向的走廊尽头有一座沙发，7、8 两个月，每当我郁闷、感到压力巨大的时候就走出实验室，一直到走廊尽头的沙发坐下，耳机里放着平静的音乐，有时祈祷，有时叹气……今后无论在哪条道路上走多远，都不会忘记那个盛夏，我曾经对计算机体系结构无限热忱……

6.2 未来展望

在我们架构的基础上，首先最显而易见的是做流水式 Cache 把取指效率翻倍，这样应当能较大地提升 IPC，同时后端因为有 ROB，做乱序发射是没问题的，并且今年比赛上科大通过引入 L2 cache，在一些访存压力比较大的性能测试点上取得了非常好的效果（一举击败清华夺得特等，128KB 的 L2 大缓存能把所有测试数据都兜住了），因此如果要在我们的架构上优化的话，我推荐优先提升前端效率，其次是认真做好 L2 Cache（这个缓存一致性问题还是很难解决的），最后是后端改出乱序，其中 L2 Cache 需要耗费较大的精力但是性能测试中的回报一定是最高的，而乱序是非常难调且收益不稳定但是成就感最高的哈哈。

6.3 备赛经验及建议

其实和备赛相关的经验在前文偶有提及，这里系统性总结一下，我认为我们的整个路径对后来人的参考意义是非常大的，如果你们要从零开始，那么跟着我们的路径实现可以少踩很多雷。

我们从寒假开始备赛，首先学习体系结构的基础知识——计算机组成原理，我们跟着刘宏伟老师的课程学习，这个课程太过抽象，这里我推荐你们采用《软硬件接口》这本书，用 MIPS 版本还是 RV 版本随意，不必全部学完，只需要把书中关于流水线的部分全部认真学完，还有关于了解 CPU 在执行过程中的异常就行。

然后跟着《CPU 设计实战》这本书慢慢做，注意一定要选择 LoongArch 版本（这本书还有 MIPS 版本，不推荐），如果不跟着《实战》做，直接对着指令手册实现功能，那么极大可能被指令手册中一些表述不清的地方误导，23 年 Invalid Syntax 和 24 年春日影队都被这个问题困扰过。《实战》的编者毕竟就是龙架构首席设计师，对 LA32R 中很多特权态的功能都做了更详细的阐述，配合着 openla500

一边看代码一边看书就能很好把握特权态指令的功能，少走很多很多弯路。

《实战》一开始的几个实验在网上（譬如 CSDN）都能找到答案，不妨参考这些答案加快进度。完成所有用户态指令并且写完前递之后就可以开始分工了，在此之前最好所有的队员都跟着《实战》做一遍。

完成《实战》中有关精确异常相关内容后，接下来是写类 SRAM 总线。此时虽然还没有加入 cache，但是类 SRAM 总线的约定可以作为流水线和 Cache 之间数据交互的规定。然后是在没有 cache 的情况下添加 AXI 总线的支持，不过此时相当于直接从流水线的类 SRAM 总线设计转接桥连接 AXI 接口，实际上这没有必要，因为之后所有的访存都要通过 cache 与总线进行交互，所以不妨在做完类 SRAM 总线后直接开始实现 cache+AXI 转接桥，虽然一步到位比较困难但是可以避免重复的精力消耗（提示一下实现 cache 之后 AXI 转接桥可以直接把 openla500 的拿过来用，我们就是这样干的）。

做完设计实战之后继续通过 chiplab func19，此时就可以分两条线，用五级流水实现系统启动，以及在此基础上改出超标量架构。

此时你可能会有点疑问，五级流水能启动系统，但是改了架构之后岂不是要重新调试一遍吗？nonono，之前提到我们把 CPU 分为流水线和访存两个子系统，如果 CPU 能启动系统说明访存以及异常处理这块基本上没有问题，而我们修改完新架构之后肯定要跑性能测试，能通过性能测试的情况下又说明新架构的流水线其实没有特别大的问题，如果只是改动前后端，基本上不会对以 Cache 为主体的访存系统做太大修改。那么新架构完成之后相当于流水线和访存基本上都没什么大的缺陷，此时用新架构启动系统是非常快速的，实际上也确实如此，我们在初赛提交之前还没有尝试用双发的架构启动系统，初赛提交之后只花了两天就成功启动了。

现在回过头来看，我们似乎每一步都走对了，一开始选择了最贴合 LA32R 的《实战》来入门，进入 7 月后又很快接入了 chiplab 进行测试，并且直接奔着启动系统去，初赛到复赛的性能优化也是挑了几个最容易收益也最大的地方。现在想想，如果当初不沿着这条路走那么根本不可能在比赛之前完成这么大的工作量。

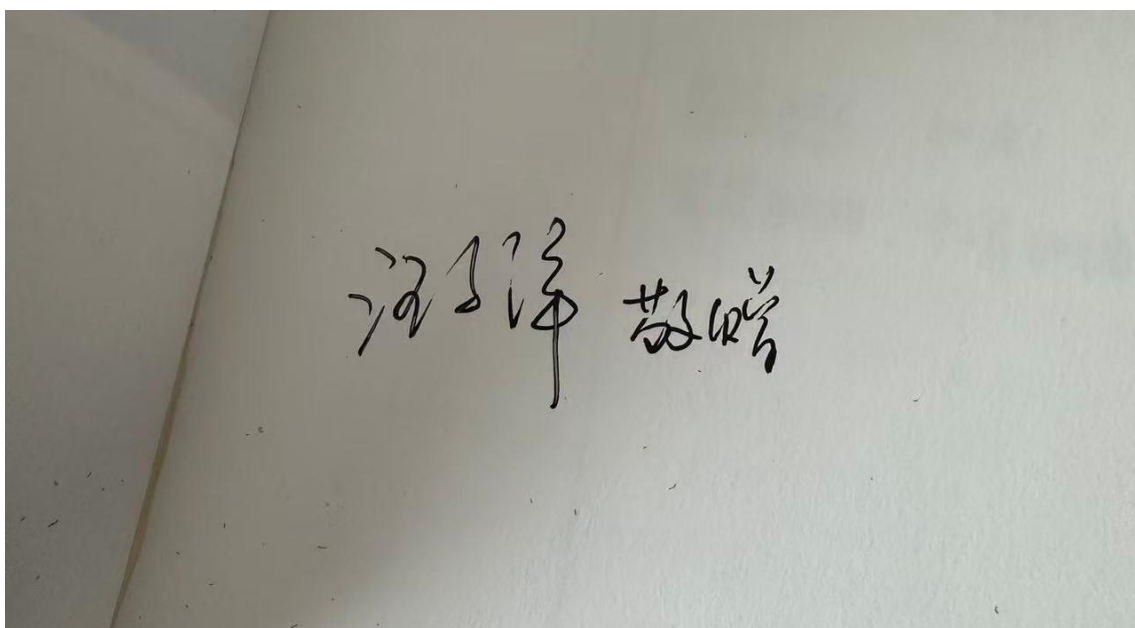
附录——夹带一些私货



在 T2 6 楼的尽头，坐在沙发上静静看着主楼的肩膀……



我们在哈尔滨的住处，楼层非常高！



汪老师的签名！珍藏！



主楼！每个哈工大人的魂！



2025年全国大学生计算机系统能力大赛CPU设计赛（龙芯杯）全国总决赛

东北林业大学 2025.08.25



中国芯任重道远……只争朝夕！